



Instituto Tecnológico y de Estudios Superiores de Monterrey

Escuela de Ingeniería y Ciencias

Ingeniería en Ciencia de Datos y Matemáticas

**Software Development Kit (SDK)
Gestor de Identidades y Correo Firmado**

Equipo de Trabajo 2 - Grupo 603:

Valeria Arciga Valencia - A01737555
Diego Octavio Arias Incháustegui - A00838285
Alberto Boughton Reyes - A01178500
Henning Arvid Ladewig - A01831983
Ximena Montes Bautista - A01737949
Fernando Daniel Saucedo Hernández - A01385490

Profesores del bloque:

Raúl Gómez Muñoz, Daniel Otero Fadul,
Alberto Francisco Martínez Herrera, Pedro Leonel Olaya Trejos

Socio Formador:

Casa Monarca Ayuda Humanitaria al Migrante A.B.P.

Monterrey, Nuevo León
14 de junio de 2026

Índice

1. Propósito del SDK	4
2. Visión general del sistema	4
2.1. Objetivos funcionales	4
2.2. Arquitectura lógica	4
2.3. Estructura de Archivos	6
3. Gestión de identidades y política de control de acceso	8
4. Desarrollo de BackEnd	9
4.1. Módulo de inicialización (<code>run.py</code>)	9
4.2. Interfaz de programación de aplicaciones (<code>api_intf.py</code>)	9
4.3. Módulo de autenticación (<code>auth.py</code>)	11
4.4. Módulo de criptografía sobre certificados (<code>ccore.py</code>)	11
4.5. Módulo de criptografía sobre correos (<code>mail_crypto.py</code>)	11
4.6. Bootstrap del administrador inicial	12
4.7. Capa de datos (<code>db.py</code>)	12
5. Desarrollo de FrontEnd	13
5.1. Pantalla de inicio de sesión (<code>login.html</code>)	13
5.1.1. Pantalla de cambio obligatorio de contraseña (<code>change_password.html</code>) . . .	14
5.1.2. Panel administrativo (<code>index.html</code>)	14
5.1.3. Panel de correos firmados	14
5.1.4. Página pública de verificación (<code>verify.html</code>)	15
6. Conexión y flujos de operación	16
6.1. Flujos de operación principales	16
6.1.1. Registro de nuevos usuarios	16
6.1.2. Registro de certificados emitidos externamente	17
6.1.3. Autenticación y cambio obligatorio de contraseña	19
6.1.4. Firma y verificación pública de correos institucionales	20
7. Stack tecnológico	22
8. Estructura del SDK	22
9. Modelo de dominios	23
9.1. Usuario	23
9.2. Departamentos soportados	23
9.3. Niveles de acceso	23
9.4. Certificado	23
9.5. Registro de autenticación	24
9.6. Correo firmado	24
9.7. Adjunto firmado	24
10. Persistencia y serialización	24
10.1. Serialización de fechas	25
10.2. Serialización de certificados	25
10.3. Tablas TinyDB	25

11.Criptografía aplicada	25
11.1. Módulo <code>ccore.py</code>	25
11.2. Esquema criptográfico	25
11.3. Módulo <code>mail_crypto.py</code>	25
11.4. Canonicalización del cuerpo	26
12.Autenticación	26
12.1. Módulo <code>auth.py</code>	26
12.2. Flujo de autenticación	26
12.3. Primer acceso	26
12.4. Bootstrap administrativo	26
13.Autorización RBAC	27
13.1. Matriz de permisos	27
13.2. Reglas específicas	27
14.Contratos de API	27
14.1. Convenciones generales	27
14.2. Endpoint <code>POST /auth/login</code>	27
14.3. Endpoint <code>GET /auth/me</code>	28
14.4. Endpoint <code>POST /auth/change-password</code>	28
14.5. Endpoint <code>GET /users</code>	28
14.6. Endpoint <code>POST /users</code>	29
14.7. Endpoint <code>PATCH /users</code>	29
14.8. Endpoint <code>DELETE /users</code>	29
14.9. Endpoint <code>GET /certs</code>	29
14.10Endpoint <code>POST /certs</code>	29
14.11Endpoint <code>GET /certs</code>	30
14.12Endpoint <code>GET /mail</code>	30
14.13Endpoint <code>POST /mail</code>	30
14.14Endpoint <code>POST /mail/x509</code>	31
14.15Endpoint <code>GET /mail/<messageid>/verify</code>	31
14.16Endpoint <code>GET /public/verify</code>	31
15.Frontend SDK	32
15.1. Estado local	32
15.2. Guardia de autenticación	32
15.3. Cliente HTTP genérico	32
15.4. Firma en navegador	32
16.Flujos principales	33
16.1. Flujo A: alta de usuario	33
16.2. Flujo B: carga o generación de certificado	33
16.3. Flujo C: login y sesión	33
16.4. Flujo D: correo firmado interno/externo	33
16.5. Flujo E: verificación pública	34
17.Ejemplos de integración en Python	34
17.1. Login	34
17.2. Crear usuario	34
17.3. Consultar certificados vigentes	34
17.4. Verificar un correo por ID	34

18.Ejemplos de integración en JavaScript	35
18.1. Consumir la API autenticada	35
18.2. Login en JavaScript	35
19.Manejo de errores	35
19.1. Errores de validación típicos	35
19.2. Patrones recomendados para integradores	35
20.Seguridad y mejores prácticas	36
21.Limitaciones actuales y extensiones sugeridas	36
Referencias ³⁷	
22.Apéndice: esquema resumido de endpoints	37
23.Apéndice: recomendaciones para Overleaf	37

1. Propósito del SDK

Este SDK documenta de manera integral la plataforma Gestor de Identidades Casa Monarca; una solución web orientada a la administración de identidades, certificados X.509, autenticación con JWT y correo firmado digitalmente. El objetivo del documento es servir como referencia técnica para desarrollo, integración, mantenimiento, auditoría y extensión funcional del sistema.

El SDK cubre arquitectura, dependencias, módulos, modelos de datos, contratos de API REST, flujos de autenticación, reglas de autorización, persistencia en TinyDB, criptografía aplicada, firma y verificación de correos, integración desde cliente web y lineamientos de seguridad operativa. Está redactado para ser pegado directamente en Overleaf y compilarse como documentación formal del proyecto.

2. Visión general del sistema

La solución está compuesta por un backend en Python con Flask/Flask-RESTX, una capa de persistencia en TinyDB y un frontend monolítico en `index.html` con HTML, CSS y JavaScript. El sistema implementa autenticación basada en JWT, control de acceso RBAC por niveles jerárquicos, emisión y consulta de certificados X.509, y un módulo de correo firmado digitalmente con verificación autenticada y verificación pública de un solo uso.

2.1. Objetivos funcionales

- Administrar usuarios institucionales con niveles de acceso y departamento.
- Asociar credenciales criptográficas a perfiles autorizados.
- Emitir, registrar y consultar certificados X.509.
- Proteger el acceso a la plataforma con credenciales y tokens JWT.
- Permitir firma digital de correos desde el navegador sin exponer la clave privada al servidor.
- Generar enlaces efímeros de validación pública para destinatarios externos.
- Aplicar el principio de mínimo privilegio mediante RBAC.

2.2. Arquitectura lógica

1. **Presentación:** interfaz web administrativa y de operación en `index.html`.
2. **API:** endpoints REST definidos principalmente en `api_intf.py`.
3. **Persistencia:** almacenamiento de usuarios, certificados, credenciales de autenticación y correos en TinyDB mediante `db.py`.
4. **Criptografía:** creación de certificados en `ccore.py`, autenticación en `auth.py` y utilidades de firma/verificación de correo en `mail_crypto.py`.

En la Figura 1 se detalla un diagrama de la arquitectura propuesta, en donde se observa la conexión entre el FrontEnd, BackEnd y el almacenamiento, incluyendo los módulos de autenticación y firma de correos incorporados en la presente etapa.

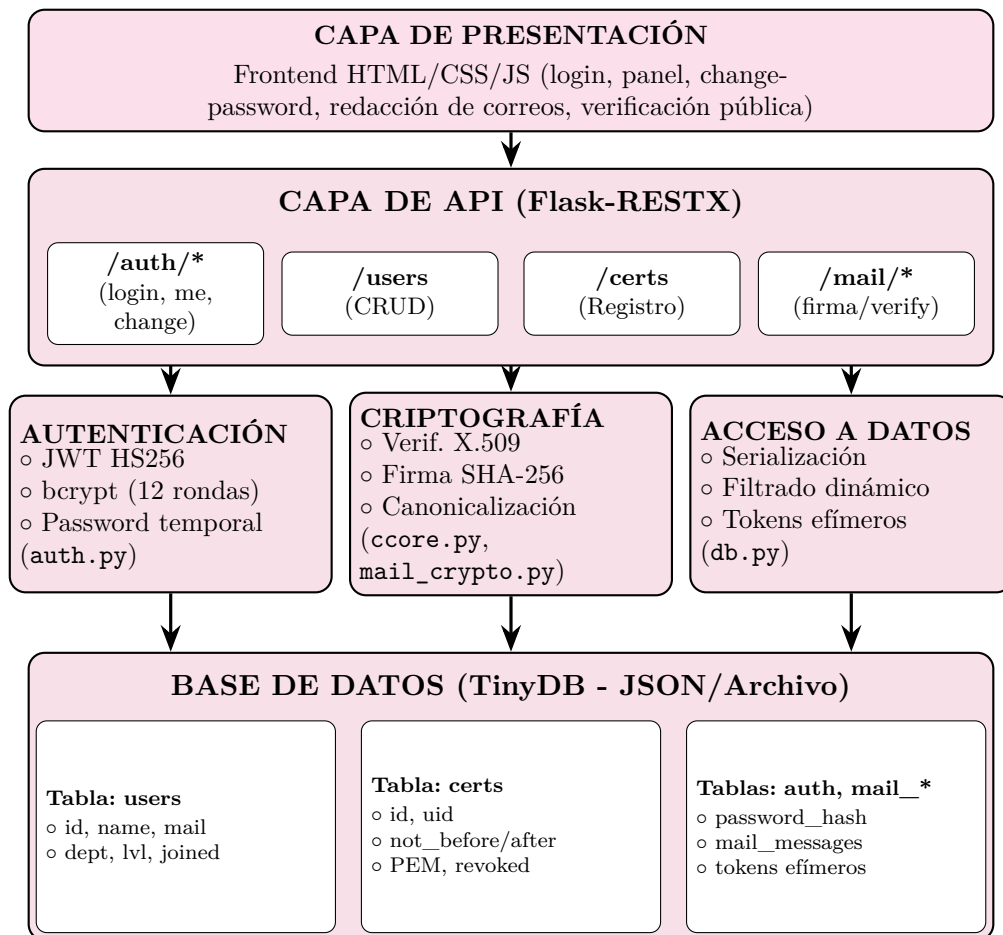


Figura 1: Arquitectura del sistema en capas con los módulos de autenticación y firma de correos incorporados (elaboración propia).

Paquete	Versión	Descripción Operativa
flask	1.1.2	Framework web ligero para la gestión del servidor HTTP.
flask-restx	0.3.0	Extensión Flask para la especificación y validación de endpoints REST.
flask-swagger-ui	3.36.0	Interfaz interactiva embebida para la documentación de la API.
flask-cors	3.0.10	Mecanismo para el control de acceso entre orígenes cruzados.
tinydb	4.4.0	Base de datos embebida basada en archivos JSON para persistencia local.
cryptography	3.4.7	Operaciones criptográficas avanzadas: X.509, RSA, SHA-256 y verificación de firmas.
bcrypt	3.1.7	Hashing adaptativo de contraseñas con factor de costo configurable.
PyJWT	1.7.1	Emisión y verificación de tokens JWT firmados con HS256.
python-dotenv	0.20.0	Carga de variables de entorno desde archivo <code>.env</code> aislado del repositorio.
validators	0.18.2	Componente para la validación estricta de sintaxis de correos electrónicos y datos.

Cuadro 1: Dependencias del sistema y descripción de paquetes de software (elaboración propia).

2.3. Estructura de Archivos

Para garantizar el mantenimiento preventivo y la escalabilidad del sistema por parte del equipo de TI de la organización, el proyecto se organizó bajo una estructura de directorios modular. En la Figura 2 se detalla el árbol de archivos que componen la solución técnica.

Los directorios `examples/backend/` y `examples/tls/` contienen prototipos de exploración inicial generados durante la fase de evaluación tecnológica del proyecto. Estos archivos se conservan en el repositorio por trazabilidad histórica pero no forman parte del sistema desplegado.

Con el objetivo de auditar la solución, se ha realizado una recopilación de los componentes activos del sistema, los cuales se detallan en el Cuadro 2.

```

MA2006B_Reto/
├── src/
│   ├── api_intf.py # Definición de endpoints REST
│   ├── auth.py # JWT y hashing de contraseñas
│   ├── ccore.py # Verificación de certificados X.509
│   ├── db.py # Capa de persistencia (TinyDB)
│   └── mail_crypto.py # Firma y verificación de correos
├── templates/
│   ├── index.html # Panel administrativo
│   ├── login.html # Inicio de sesión
│   ├── change_password.html # Cambio obligatorio de contraseña
│   └── verify.html # Verificación pública de correos
├── examples/
│   ├── create_dummy_data.py # Inicialización de datos de prueba
│   ├── certutil.py # Utilidades de generación de
│   │               certificados
│   ├── gen_cert.py # Emisión de certificados para usuarios
│   ├── backend/ # Prototipos de exploración inicial
│   └── tls/ # Prototipos de exploración inicial
├── run.py # Punto de entrada del servidor
├── setup.py # Configuración del paquete
├── setup_auth.py # Generación inicial del archivo .env
├── requirements.txt # Dependencias
├── README.md # Documentación
└── .gitignore # Exclusiones git

```

Figura 2: Estructura de directorios y componentes del proyecto (elaboración propia).

Componente	Tecnología	Responsabilidad de Software
run.py	Flask 1.1.2	Punto de entrada: parsea argumentos, valida archivos del servidor, registra rutas HTML y monta la API REST bajo el prefijo /api.
api_intf.py	flask-restx	Definición de endpoints, validación de esquemas, control de autorización por rol y orquestación del bootstrap del administrador inicial.
auth.py	PyJWT, bcrypt	Emisión y verificación de tokens JWT, hashing de contraseñas con bcrypt, y generación de contraseñas temporales criptográficamente seguras.
ccore.py	cryptography	Extracción de datos de identidad de certificados X.509 registrados y verificación de cadenas de firma contra la lista de administradores.
mail_crypto.py	cryptography	Canonicalización de cuerpos de correo, firma RSA-PKCS1v15 con SHA-256, verificación de firmas y cálculo de huellas digitales de certificados.
db.py	TinyDB 4.4.0	Capa de persistencia para usuarios, certificados, registros de autenticación, mensajes firmados y tokens efímeros de verificación.
templates/*.html	HTML / CSS / JS, jsrsasign	Capa de presentación: panel administrativo, autenticación, cambio de contraseña, redacción de correos con firma local en navegador y página pública de verificación.

Cuadro 2: Distribución de componentes del sistema y tecnologías (elaboración propia).

3. Gestión de identidades y política de control de acceso

El diseño de la solución contempla la dinámica de rotación de colaboradores en Casa Monarca, estructurando un modelo jerárquico para categorizar a los usuarios. Este modelo se implementó en el código a través de la enumeración `LevelEnum`, en donde se clasifican las identidades determinando qué perfiles requieren credenciales criptográficas y estableciendo sus límites de operación.

De acuerdo con las reglas de negocio establecidas para la Organización Socio Formadora, el flujo de la información exige un control estricto sobre quién registra, revisa y modifica los datos de los beneficiarios, así como quién posee la facultad de emitir comunicaciones oficiales en nombre de la institución. Para abordar estas dos dimensiones, el sistema modela un Control de Acceso Basado en Roles (RBAC) fundamentado tanto en las operaciones de bases de datos CRUD (Crear, Leer, Actualizar, Eliminar) como en las capacidades de firma criptográfica.

A nivel de código, esta jerarquía se encuentra parametrizada a través del enumerador `LevelEnum`, estableciendo cuatro niveles con capacidades escalonadas:

1. **Nivel 1 — Administrador:** Posee control absoluto sobre el sistema, incluyendo la gestión completa del padrón de usuarios, la emisión de certificados, la revocación y la firma de comunicaciones institucionales.
2. **Nivel 2 — Coordinador:** Personal con autoridad para la gestión de su área. El código refleja la segmentación de este nivel mediante el enumerador `DeptEnum`, mapeando las áreas operativas de Casa Monarca. Los coordinadores poseen capacidades de firma de correos institucionales.
3. **Nivel 3 — Operativo:** Personal enfocado en la consulta de información personal y el ingreso de datos demográficos básicos. No participa en flujos criptográficos.
4. **Nivel 4 — Captura:** Personal de registro con acceso restringido a su propio perfil. No participa en flujos criptográficos.

A continuación, en el Cuadro 3, se sintetiza la matriz de acceso CRUD sobre el padrón de usuarios para cada perfil del sistema. Esta matriz se aplica simétricamente tanto en la capa de presentación (donde las opciones no disponibles se ocultan o deshabilitan) como en la capa de lógica del backend, que valida cada solicitud y rechaza con un código HTTP 403 cualquier operación no autorizada según el rol del solicitante.

Nivel	Perfil	Crear (C)	Leer (R)	Actualizar (U)	Eliminar (D)	Alcance Operativo
Nivel 1	Administrador	Cualquier nivel	Todo	Cualquier usuario		Gestión total.
Nivel 2	Coordinador	Solo N3 y N4	Todo	N3 y N4 del mismo depto.	×	Gestión segmentada por departamento.
Nivel 3	Operativo	×	Perfil propio	×	×	Consulta de perfil propio.
Nivel 4	Captura	×	Perfil propio	×	×	Consulta de perfil propio.

Cuadro 3: Matriz de permisos CRUD sobre el padrón de usuarios (elaboración propia).

Adicionalmente, dado que la firma criptográfica constituye una facultad institucional con implicaciones legales (no repudio), el sistema diferencia explícitamente qué niveles pueden emitir comunicaciones oficiales y revocar credenciales. Esta segmentación se sintetiza en el Cuadro 4.

Nivel	Perfil	Recibe certificado	Firma correos	Revoca certificados
Nivel 1	Administrador	✓	✓	✓
Nivel 2	Coordinador	✓	✓	×
Nivel 3	Operativo	×	×	×
Nivel 4	Captura	×	×	×

Cuadro 4: Matriz de capacidades criptográficas y firma institucional por nivel de acceso (elaboración propia).

Esta segmentación implementa el principio de mínimo privilegio descrito en el marco de referencia [Stallings, 2017, Zúñiga and Hecht, 2023]: cada usuario recibe únicamente las capacidades estrictamente necesarias para su rol institucional. La concentración de capacidades criptográficas en los niveles N1 y N2 responde a una decisión deliberada de Casa Monarca, según la cual la facultad de emitir comunicaciones oficiales con valor probatorio queda reservada al personal directivo y de coordinación, mientras que el personal operativo y de captura interactúa con el sistema exclusivamente para tareas de consulta o ingreso de datos básicos.

4. Desarrollo de BackEnd

El servidor central (BackEnd) es el núcleo lógico y de seguridad del sistema. Su desarrollo se estructuró en seis módulos fundamentales para garantizar una separación adecuada de responsabilidades: orquestación e inicialización del servidor, interfaz REST de comunicación, autenticación y gestión de sesiones, operaciones criptográficas sobre certificados, criptografía de firma de correos, y persistencia de datos.

A continuación, se detalla la implementación técnica de cada componente.

4.1. Módulo de inicialización (`run.py`)

Este componente es el punto de entrada de la aplicación. Su responsabilidad principal consiste en parsear los argumentos de línea de comandos (archivo de base de datos, certificado del servidor y llave privada del servidor), validar la existencia de los tres archivos antes de arrancar, configurar el entorno Flask y registrar las rutas del sistema.

El servidor expone dos espacios de URL diferenciados: las rutas HTML (`/`, `/login`, `/change-password`, `/verify/<token>`) que sirven las plantillas de la capa de presentación, y el espacio `/api/*` donde reside la API REST gestionada por la extensión flask-restx. La documentación interactiva autogenerada se expone bajo `/api/docs` mediante Swagger UI.

En cuanto a la configuración de red, se habilitó el soporte para el intercambio de Recursos de Origen Cruzado (CORS), permitiendo la comunicación fluida con la capa de presentación. La terminación de las conexiones TLS se delega al servidor frontal del hosting (HostGator), el cual soporta nativamente certificados de Let's Encrypt según se documenta en la matriz de compatibilidades. Esta delegación responde a la práctica estándar de despliegue de aplicaciones Flask en entornos de alojamiento compartido, donde la capa de aplicación opera tras un proxy inverso que termina las conexiones cifradas.

4.2. Interfaz de programación de aplicaciones (`api_intf.py`)

Este módulo define la comunicación del sistema mediante una arquitectura RESTful, utilizando la extensión flask-restx. Se encarga de procesar las solicitudes entrantes, validar la integridad de las cargas útiles (*payloads*), aplicar el control de autorización por rol, y devolver las respuestas HTTP correspondientes.

Para este módulo se programaron enumeraciones estrictas para modelar la estructura organizacional de Casa Monarca. Se definieron los departamentos y los niveles jerárquicos como se detalla en el Cuadro 5.

Estructura (Enum)	Valores Permitidos (Colección)	Propósito Operativo
DeptEnum	Humanitaria, PsicoSocial, Legal, Comunicación, Almacén, TI	Delimitación estricta de las áreas y departamentos funcionales de Casa Monarca.
LevelEnum	admin, coordinador, operativo, captura	Asignación del nivel de acceso jerárquico asociado al ciclo de vida del usuario.

Cuadro 5: Estructura de Enums y valores permitidos para el control operativo (elaboración propia).

El módulo agrupa los endpoints del sistema en cuatro familias funcionales:

- **Endpoints de identidades (/users):** implementan las operaciones CRUD completas sobre el padrón de usuarios. Las peticiones GET soportan un motor de búsqueda por expresiones regulares y rangos de fechas. Para el registro (POST), se integraron validaciones estrictas, asegurando que el formato del correo electrónico sea válido mediante la biblioteca `validators` y que este sea único en todo el padrón. El POST y el PATCH delegan a un módulo auxiliar la verificación del rol del solicitante a partir del token JWT del encabezado, rechazando con código 403 cualquier operación incompatible con la jerarquía descrita en la sección anterior. Como subproducto de la creación de un usuario, el sistema genera automáticamente una contraseña temporal criptográficamente segura y la devuelve en la respuesta para que el administrador la comparta con el nuevo colaborador.
- **Endpoints de certificados (/certs):** a diferencia del flujo de la etapa anterior, el endpoint POST de esta familia ya no genera certificados internamente, sino que recibe certificados X.509 en formato PEM emitidos externamente mediante la herramienta de línea de comandos descrita más adelante. El endpoint decodifica el PEM, delega al módulo criptográfico la extracción de los atributos del certificado, verifica que dichos atributos coincidan con un usuario registrado en la base de datos, y valida la firma del certificado contra la lista de certificados administradores registrados, rechazando cualquier certificado que no esté firmado por una Autoridad Certificadora interna reconocida. El endpoint GET soporta filtrado por identificadores, rangos de fechas, estado de revocación y vigencia.
- **Endpoints de autenticación (/auth/*):** comprenden `/auth/login` para emitir tokens JWT a partir de credenciales válidas, `/auth/me` para que el cliente consulte la información de la sesión activa, `/auth/change-password` para el cambio obligatorio o voluntario de contraseña, y `/auth/logout` para cerrar sesión del lado cliente. La validación de credenciales delega al módulo `auth.py` la verificación de contraseñas y la emisión de tokens.
- **Endpoints de correos firmados (/mail/* y /public/verify/*):** permiten redactar correos firmados (POST `/mail`), consultar el padrón de correos del sistema (GET `/mail`), verificar la firma de un mensaje específico (GET `/mail/<id>/verify`), y, especialmente, ofrecer una verificación pública para destinatarios externos sin necesidad de credenciales (GET `/public/verify/<token>`). Esta última familia se describe en mayor profundidad en el módulo `mail_crypto.py`.

4.3. Módulo de autenticación (`auth.py`)

Este módulo encapsula las funciones criptográficas y de gestión de credenciales del sistema de inicio de sesión. Se apoya en la biblioteca `bcrypt` para el resguardo de contraseñas, en `PyJWT` para la emisión y verificación de tokens JWT, y en `python-dotenv` para la carga de la llave secreta desde un archivo de configuración aislado del repositorio.

El hashing de contraseñas utiliza el algoritmo `bcrypt` con un factor de costo configurable de 12 rondas. Este valor establece $2^{12} = 4096$ iteraciones internas del algoritmo, equilibrando seguridad y rendimiento sobre el hardware estándar del entorno de despliegue: cada operación de verificación toma aproximadamente entre 50 y 200 milisegundos, una latencia imperceptible para el usuario legítimo pero suficiente para encarecer drásticamente los ataques de fuerza bruta ante un eventual compromiso de la base de datos.

Para la generación de contraseñas temporales asignadas a nuevos usuarios durante el registro, el módulo utiliza el módulo `secrets` de la biblioteca estándar de Python sobre un alfabeto cuidadosamente seleccionado de 56 caracteres que excluye glifos visualmente ambiguos como `i`, `l`, `o`, `0`, `1`, `I`, `L` y `O`. Esta selección reduce el riesgo de errores cuando el administrador comunica la contraseña al usuario por canales no electrónicos. Las contraseñas resultantes tienen una longitud fija de 12 caracteres, lo que proporciona un espacio de búsqueda de $56^{12} \approx 5.7 \times 10^{20}$ combinaciones posibles, suficiente para resistir cualquier intento de adivinación durante el corto período de vida útil de la credencial temporal.

La emisión de tokens JWT se realiza con el algoritmo HS256 y una vigencia de 8 horas desde la emisión. El *payload* del token incluye el identificador del usuario, su nivel jerárquico, su departamento, y las marcas temporales de emisión y expiración. Aunque el token transporta el nivel y departamento del usuario, los endpoints sensibles del sistema realizan adicionalmente una consulta al padrón para obtener el estado actual del usuario, evitando así que un cambio de rol o departamento del lado servidor quede invalidado únicamente al expirar el token original. La llave secreta para la firma de tokens se carga al iniciar el módulo desde la variable de entorno `JWT_SECRET`: si dicha variable no está definida, el módulo aborta con un error explícito, evitando la ejecución del servidor sin un secreto criptográfico válido.

4.4. Módulo de criptografía sobre certificados (`ccore.py`)

Este componente alberga las primitivas criptográficas relacionadas con los certificados X.509 registrados en el sistema. En la etapa anterior, este módulo era responsable de la generación interna de certificados: en la presente etapa, su rol se redefinió hacia la extracción de atributos y la verificación de cadenas de confianza, en concordancia con el flujo de emisión externa de certificados.

El módulo expone dos funciones principales. La primera, `extract_user_data`, recibe un objeto de certificado y extrae los atributos relevantes del sujeto X.509: el nombre, el correo electrónico, el identificador único del usuario y la combinación de departamento y nivel codificada en la Unidad Organizacional. Esta función actúa como puente entre los datos embebidos en el certificado y la estructura de usuario manejada por la base de datos: cuando un administrador registra un nuevo certificado en el sistema, esta función permite verificar que los atributos del documento coincidan con los del usuario al que se pretende asociar.

La segunda función, `verify_cert`, recibe un certificado y una lista de certificados de confianza (los emitidos a los administradores registrados), y verifica la firma del certificado contra cada uno de los confiables, devolviendo afirmativo si alguno valida correctamente. Esta verificación utiliza el esquema RSA-PKCS1v15 con el algoritmo de resumen embebido en el propio certificado (en el sistema actual, SHA-256), garantizando que únicamente los certificados emitidos por una Autoridad Certificadora interna reconocida puedan registrarse en el sistema.

4.5. Módulo de criptografía sobre correos (`mail_crypto.py`)

Este módulo es completamente nuevo en la presente etapa y constituye el núcleo criptográfico del sistema de firma de comunicaciones institucionales. Provee las primitivas necesarias para cano-

nicalizar mensajes, firmar contenido con llaves privadas, verificar firmas con certificados públicos, calcular huellas digitales y codificar datos binarios para su transporte en formato JSON.

La función `canonicalize_body` resuelve un problema sutil pero crítico en la firma de mensajes a través de una red heterogénea. La firma criptográfica opera sobre una secuencia de bytes específica, por lo que el cliente que firma y el servidor que verifica deben producir exactamente la misma representación binaria del contenido a firmar, hasta el último byte. Diferencias invisibles para el usuario (saltos de línea Windows CRLF frente a Unix LF, codificaciones distintas, espacios al final) pueden invalidar una firma legítima. Para evitar esta inconsistencia, la función normaliza todos los saltos de línea a LF, ensambla un encabezado fijo con los campos **To**, **Subject** y el cuerpo, asegura que el resultado termine en un salto de línea, y codifica el resultado en UTF-8. Esta canonicalización se implementa idénticamente del lado del cliente (en el navegador, mediante JavaScript) y del lado del servidor (en Python), garantizando interoperabilidad entre ambas partes.

Las funciones `sign_bytes` y `verify_bytes` encapsulan el esquema de firma RSA-PKCS1v15 con SHA-256, codificando la firma resultante en Base64 para su transporte en JSON. La función `cert_fingerprint_sha256` calcula la huella digital SHA-256 del certificado completo, valor que se almacena junto con cada mensaje firmado como identificador robusto de la credencial utilizada. Finalmente, las funciones auxiliares `sha256_bytes` y `b64decode_bytes` encapsulan operaciones rutinarias sobre archivos adjuntos.

La decisión arquitectónica más relevante del módulo es la separación entre firma del cuerpo y firma de archivos adjuntos: cada adjunto se firma de manera independiente, lo cual permite verificar la integridad de cada componente del mensaje por separado y simplifica la lógica de almacenamiento y verificación en la base de datos.

4.6. Bootstrap del administrador inicial

Para garantizar que el sistema sea inmediatamente operable tras su despliegue, el módulo `api_intf.py` incorpora una rutina de inicialización que se ejecuta al arrancar el servidor. Esta rutina verifica si la tabla de autenticación contiene algún registro: en caso negativo, busca en el padrón un usuario con el correo `admin@casamonarca.mx` y, si lo encuentra (típicamente porque fue creado por el script de datos de prueba), reutiliza su identificador. Si no existe, lo crea con los atributos correspondientes al perfil de administrador del departamento de Tecnologías de la Información. En ambos casos, el sistema asigna la contraseña inicial `admin123` con la bandera de cambio obligatorio activada, asegurando que esta credencial preasignada sea sustituida inmediatamente durante el primer inicio de sesión.

4.7. Capa de datos (`db.py`)

Este módulo administra la persistencia de la información en memoria secundaria, interactuando de manera exclusiva con el motor TinyDB. La capa expone cinco tablas: **users** (padrón de usuarios), **certs** (certificados registrados), **auth** (credenciales de autenticación), **mail_messages** (correos firmados) y **mail_attachments** (archivos adjuntos firmados independientemente).

Dado que objetos como fechas nativas de Python (`date`) o los certificados `x509.Certificate` no son compatibles directamente con estructuras JSON, se desarrollaron funciones bidireccionales de serialización y deserialización. Las fechas se transforman en tuplas de enteros (año, mes, día) y los certificados se codifican como cadenas de texto PEM para su almacenamiento seguro, reconstruyéndose en objetos nativos durante la lectura.

En términos de integridad referencial y unicidad, las funciones de escritura (`add_user` y `add_cert`) están programadas con rutinas de validación interna que generan identificadores únicos aleatorios mediante reintento, previniendo colisiones de datos. La función `add_user` valida adicionalmente la unicidad del correo electrónico antes de registrar al usuario.

La tabla **mail_messages** incorpora un mecanismo de doble cuerpo: el campo `body_raw` almacena el contenido original del correo (sobre el cual se calculó la firma), mientras que el campo `body` almacena una versión enriquecida que incluye un pie de página con la URL pública de verificación.

Esta separación es necesaria por razones criptográficas: la URL de verificación solo puede generarse después de crear el mensaje (porque depende de un token aleatorio asignado en ese momento), por lo que la firma no puede incluirla. Aun así, el sistema necesita mostrar al usuario un cuerpo que contenga el enlace para distribución. Mantener ambos campos permite verificar la firma sobre el contenido original mientras se presenta al lector un cuerpo enriquecido.

Para la verificación pública de correos por destinatarios externos, la tabla `mail_messages` incluye los campos `validation_token`, `expires_at` y `token_used`. Estos campos materializan un mecanismo de enlace efímero de un solo uso: al crear un mensaje dirigido a un destinatario externo, el sistema genera un token aleatorio de 128 bits, establece su expiración a 24 horas desde la creación, y lo marca como no consumido. La función `mark_token_used` marca el token como consumido la primera vez que se accede al enlace público, evitando reutilización del mismo.

Por último, la tabla `auth` almacena, para cada usuario, su hash bcrypt de contraseña, la bandera de cambio obligatorio, y las marcas temporales de creación y última actualización del registro. Las funciones `get_auth`, `set_auth` y `clear_must_change` encapsulan el acceso a esta tabla, manteniéndola desacoplada del resto de la lógica del sistema.

5. Desarrollo de FrontEnd

La capa de presentación fue diseñada con un enfoque centrado en la usabilidad y la eficiencia, garantizando que el personal de Casa Monarca pueda operar el sistema sin requerir conocimientos técnicos avanzados. Su construcción se basó enteramente en tecnologías web nativas (HTML5, CSS3 y JavaScript), evitando deliberadamente marcos de renderizado como React o Angular que exigirían pasos de compilación incompatibles con el contexto de mantenimiento del socio formador.

En esta etapa, la capa de presentación creció desde un único archivo a un conjunto de cuatro plantillas, cada una correspondiente a una pantalla funcional del sistema. La Tabla 6 sintetiza las plantillas y su rol.

Plantilla	Función
<code>login.html</code>	Pantalla de autenticación: captura de credenciales, validación de campos vacíos en cliente, redirección condicional según el estado del usuario.
<code>change_password.html</code>	Cambio obligatorio de contraseña en el primer acceso, con validaciones independientes sobre contraseña actual, longitud mínima y coincidencia de confirmación.
<code>index.html</code>	Panel principal autenticado: dashboard de métricas, directorio de usuarios, redacción de correos firmados y consulta de mensajes del padrón institucional.
<code>verify.html</code>	Página pública de verificación de correos firmados, accesible sin credenciales mediante un enlace efímero de un solo uso.

Cuadro 6: Plantillas que componen la capa de presentación del sistema (elaboración propia).

5.1. Pantalla de inicio de sesión (`login.html`)

La pantalla de autenticación es el único punto de entrada al panel privado del sistema. Captura el correo institucional y la contraseña del usuario, aplica una validación local previa al envío para evitar peticiones innecesarias al servidor cuando los campos están vacíos, y delega la verificación de credenciales al endpoint `POST /api/auth/login` del backend. En caso de éxito, almacena el token JWT recibido y el resumen del usuario en `sessionStorage` (más adecuado que `localStorage` por su limpieza automática al cerrar el navegador), e inspecciona la bandera `must_change_password` del response para redirigir al usuario a la pantalla de cambio obligatorio o directamente al panel

principal según corresponda. Los errores del servidor se presentan de forma diferenciada según su tipo: campos vacíos detectados localmente se señalizan resaltando los campos en rojo, mientras que las credenciales rechazadas por el backend se muestran en un mensaje descriptivo sobre el formulario.

5.1.1. Pantalla de cambio obligatorio de contraseña (`change_password.html`)

Esta pantalla implementa el flujo de transición desde una contraseña preasignada (la inicial del administrador o la temporal generada al registrar a un usuario) hacia una contraseña personal. La interfaz aplica cuatro validaciones independientes antes de enviar la solicitud al servidor: la presencia de la contraseña actual, una longitud mínima de ocho caracteres en la nueva contraseña, que la nueva contraseña sea distinta de la actual, y que el campo de confirmación coincida con la nueva contraseña. Cada validación falla con un mensaje específico sobre el campo correspondiente, evitando la confusión típica de un único mensaje genérico. La política de longitud mínima de ocho caracteres sin requerimientos de composición se alinea con las recomendaciones modernas del NIST SP 800-63B [Grassi et al., 2017], que desaconsejan reglas de complejidad arbitrarias por favorecer contraseñas predecibles. Tras el cambio exitoso, el sistema actualiza el hash bcrypt en la base de datos, elimina la bandera de cambio obligatorio y redirige al panel principal.

5.1.2. Panel administrativo (`index.html`)

El panel principal centraliza la operación cotidiana del sistema para los niveles administrativo y de coordinación. Al cargarse, ejecuta una rutina de verificación de sesión que confirma la presencia de un token JWT válido en `sessionStorage`, y en caso contrario, redirige al usuario a la pantalla de inicio de sesión antes de renderizar contenido alguno. Para evitar el parpadeo visual asociado al renderizado previo a la verificación, el cuerpo del documento se carga inicialmente con visibilidad oculta y se revela solo tras confirmar la autenticación. Todas las peticiones subsecuentes al backend se canalizan a través de una función auxiliar (`apiFetch`) que adjunta automáticamente el token JWT al encabezado de autorización y centraliza el manejo de respuestas: ante un código 401, la función limpia la sesión local y redirige nuevamente al inicio de sesión.

El panel se organiza en torno a varios componentes funcionales. En primer lugar, el tablero de control superior expone cuatro tarjetas estadísticas con métricas en tiempo real del sistema: total de usuarios registrados, certificados vigentes, certificados revocados y certificados próximos a expirar en un horizonte de 30 días. En segundo lugar, una tabla dinámica del directorio de usuarios permite la búsqueda por coincidencia de texto, el filtrado por nivel de acceso y el filtrado por estado del certificado, actualizando el Modelo de Objetos del Documento (DOM) sin recargar la página. En tercer lugar, un conjunto de modales superpuestos aísla las operaciones críticas: el modal de creación con su formulario de registro, el modal de edición con datos precargados, el visor de certificados con el detalle X.509 y la barra de progreso de vigencia, el modal especializado que presenta la contraseña temporal generada automáticamente al registrar a un nuevo usuario, y el modal de carga de certificado por arrastrar y soltar para asociar archivos PEM emitidos externamente.

Para la aplicación de la política de control de acceso, el panel implementa una función centralizada `canDo(acción, usuarioObjetivo)` que evalúa si la operación solicitada es compatible con el rol del usuario en sesión y, cuando aplica, con el del usuario objetivo. Esta función actúa como fuente única de verdad para la visibilidad de botones, opciones del selector de niveles, columnas de la tabla y secciones completas de la interfaz. Cuando el usuario en sesión pertenece al nivel operativo o de captura, el panel se reconfigura automáticamente para mostrar únicamente su propio perfil, ocultando las estadísticas globales y los controles de gestión.

5.1.3. Panel de correos firmados

Incluido dentro de `index.html` como sección lateral del panel principal, el módulo de correos firmados permite a los usuarios autorizados (administradores y coordinadores) redactar comunicaciones institucionales con firma digital y consultar los mensajes registrados en el sistema. La tabla

principal del módulo muestra el remitente con su identificador de dirección (ENVIADO o RECIBIDO), el destinatario, el asunto, la fecha, el estado de la firma evaluado por el backend, el número de adjuntos, una porción de la huella digital del certificado firmante, y los controles de acción. El alcance de visualización se ajusta automáticamente según el rol: el administrador visualiza todos los correos del sistema, mientras que el coordinador accede únicamente a los mensajes en los que participa como remitente o destinatario.

El formulario de redacción integra la composición del mensaje y la firma criptográfica en un único flujo. El usuario captura el destinatario, asunto y cuerpo del mensaje, opcionalmente adjunta archivos, y pega su llave privada en formato PEM en el área correspondiente. A medida que el usuario escribe el destinatario, la interfaz consulta el padrón local de usuarios y muestra una etiqueta diferenciada que indica si se trata de un destinatario interno (registrado en el gestor) o externo (cualquier otro correo). Esta distinción anticipa el comportamiento posterior del sistema: los correos a destinatarios externos generan un enlace efímero de verificación, mientras que los internos se verifican exclusivamente desde el panel autenticado.

La firma criptográfica ocurre íntegramente en el navegador del firmante mediante la biblioteca `jsrsasign`, que provee primitivas de criptografía RSA compatibles con los esquemas estándar utilizados por el backend. La llave privada introducida por el usuario nunca se transmite al servidor: el navegador la utiliza localmente para calcular las firmas RSA-PKCS1v15 con SHA-256 sobre el cuerpo canonicalizado y sobre cada archivo adjunto, y posteriormente envía al servidor el contenido del mensaje junto con las firmas resultantes en formato Base64. Esta arquitectura materializa el principio criptográfico de no centralización de secretos: la facultad de firmar en nombre de un usuario permanece exclusivamente bajo su control físico de la llave privada.

La canonicalización del cuerpo del mensaje implementada en JavaScript es una réplica exacta byte por byte de la función equivalente del módulo `mail_crypto.py` del backend: ambas normalizan los saltos de línea a Unix LF, ensamblan un encabezado con los campos `To` y `Subject`, aseguran que el resultado termine en un salto de línea y lo codifican en UTF-8. Esta replicación garantiza que la firma calculada en el navegador pueda ser verificada por el backend sin discrepancias originadas en diferencias invisibles de representación.

5.1.4. Página pública de verificación (`verify.html`)

La página pública de verificación constituye la interfaz que el sistema ofrece a destinatarios externos para constatar la autenticidad de los correos firmados que reciben. Es la única pantalla del sistema accesible sin credenciales, y se carga al abrir un enlace de la forma `/verify/<token>`, donde el token es el identificador efímero generado al enviar el mensaje original.

Al cargarse, la página extrae el token de la URL e invoca al endpoint público `GET /api/public/verify/<token>`. El backend ejecuta la verificación criptográfica completa: valida la firma del cuerpo del mensaje contra el certificado registrado del firmante, verifica la vigencia y el estado de revocación del certificado, marca el token como consumido para impedir reutilización, y devuelve un resumen estructurado con los resultados. La página renderiza este resultado en una tarjeta dividida en secciones: un banner superior con el veredicto general (firma válida o inválida), un bloque de estado de verificación con indicadores individuales para la integridad del mensaje y la vigencia del certificado, los metadatos del mensaje (asunto, destinatario, fecha de creación, identificador del firmante) y la huella digital del certificado firmante. En caso de token expirado, ya consumido o no encontrado, la página despliega un mensaje específico sin exponer información alguna del mensaje original, protegiendo la confidencialidad del contenido ante accesos no contemplados.

La verificación criptográfica de esta página se ejecuta del lado del servidor, no del cliente. Esta decisión de diseño prioriza la accesibilidad para destinatarios externos: el receptor puede confirmar la autenticidad de un correo desde cualquier navegador sin necesidad de descargar el certificado público de la Autoridad Certificadora interna, instalar software adicional o conocer los detalles del protocolo de firma. Como contrapartida, esta arquitectura sitúa la confianza en el servidor de Casa Monarca como verificador autoritativo (una iteración futura del sistema podría complementar este flujo con verificación opcional del lado cliente, descargando el certificado raíz y validando localmente

la firma para usuarios que requieran independencia verificativa).

6. Conexión y flujos de operación

La interconexión y la orquestación del sistema operan mediante un modelo desacoplado de transferencia de estado representacional (REST). La comunicación externa entre la capa de presentación (FrontEnd) y la lógica de negocio (BackEnd) se realiza de forma asíncrona mediante peticiones HTTP estructuradas en formato JSON, garantizando un acoplamiento débil entre componentes. Internamente en el servidor, los módulos de la API, los componentes criptográficos y la capa de persistencia se integran de manera síncrona mediante llamadas nativas de funciones.

Con el objetivo de mitigar el ingreso de datos corruptos y garantizar la consistencia operacional, el diseño implementa una estrategia de validación ejecutada tanto en la frontera del cliente como en la del servidor. Esta doble validación cumple dos roles complementarios: del lado del cliente, ofrece retroalimentación inmediata al usuario y evita peticiones innecesarias al servidor. Del lado del servidor, por otro lado, garantiza la integridad de los datos persistidos incluso ante clientes manipulados o ataques directos al API.

A continuación, se documentan los cuatro flujos operacionales principales del sistema: el registro de identidades con generación automática de credenciales temporales, el registro de certificados emitidos externamente, la autenticación de usuarios con cambio obligatorio de contraseña, y la firma y verificación pública de comunicaciones institucionales.

6.1. Flujos de operación principales

6.1.1. Registro de nuevos usuarios

El proceso inicia en la capa de presentación cuando el administrador o coordinador introduce los datos del nuevo colaborador en el formulario modal de registro. Antes de iniciar la transmisión, el motor cliente ejecuta validaciones de primera línea: campos obligatorios no vacíos, sintaxis válida del correo electrónico y unicidad local (verificada contra la lista de usuarios cargada en memoria). Si todas las validaciones aprueban, el cliente serializa los datos y envía una petición POST al endpoint `/api/users` acompañada del token JWT del usuario en sesión.

El backend, mediante el módulo `api_intf.py`, recibe la petición, decodifica el token para identificar al solicitante y valida que su nivel jerárquico autorice la operación: el administrador puede crear cualquier nivel, el coordinador únicamente niveles operativo y captura, y los demás roles reciben un rechazo con código 403. Validado el permiso, el módulo verifica la sintaxis del correo mediante la biblioteca `validators` y delega la persistencia a `db.add_user`, que valida la unicidad global del correo, genera un identificador único aleatorio y registra al usuario en la tabla correspondiente.

Como subproducto del registro exitoso, el módulo `auth.py` genera automáticamente una contraseña temporal mediante un generador criptográficamente seguro, calcula su hash bcrypt, y registra la credencial en la tabla `auth` con la bandera de cambio obligatorio activada. La respuesta al cliente incluye el identificador del nuevo usuario y la contraseña temporal en texto plano: este es el único momento del ciclo de vida del sistema en el que dicha contraseña existe en forma legible. La interfaz presenta esta contraseña al administrador mediante un modal especializado con un único botón de copiado, y advierte que tras cerrar la ventana la contraseña no podrá recuperarse.

En la Figura 3 se ilustra de manera lineal el proceso síncrono que se desencadena al dar de alta una nueva entidad dentro del endpoint de usuarios del sistema.

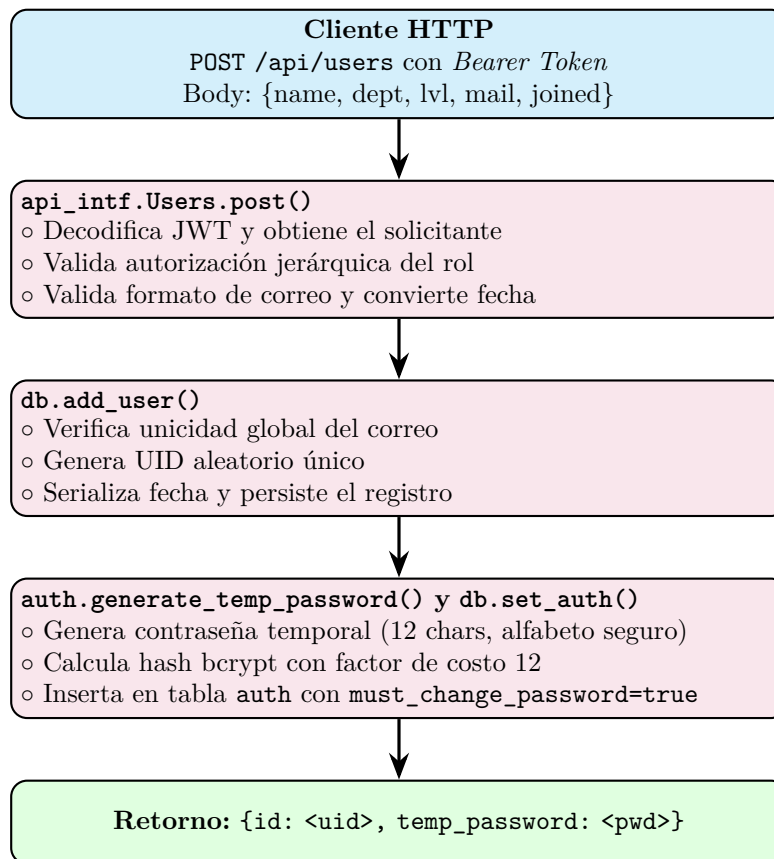


Figura 3: Diagrama de flujo: registro de nuevos usuarios con generación automática de contraseña temporal (elaboración propia).

6.1.2. Registro de certificados emitidos externamente

A diferencia del flujo de la etapa anterior, en la cual el backend generaba certificados internamente, en esta etapa los certificados se emiten fuera del sistema mediante la herramienta de línea de comandos `examples/gen_cert.py`, ejecutada localmente por el administrador. La herramienta carga la llave privada y el certificado de la Autoridad Certificadora interna, busca al usuario destinatario en la base de datos, construye el sujeto X.509 con los atributos del usuario (incluyendo el identificador y la combinación de departamento y nivel codificada en la Unidad Organizacional), genera un par de llaves RSA con tamaño configurable (mínimo 2048 bits), firma el certificado con la llave de la CA, y produce dos archivos PEM: el certificado público y la llave privada del usuario. La llave privada debe entregarse al usuario por canal separado y nunca transita por el servidor.

Una vez emitido el certificado, el administrador lo registra en el sistema mediante el modal de carga por arrastrar y soltar que se presenta tras el alta del usuario. El cliente envía el contenido PEM al endpoint `POST /api/certs`, donde el backend ejecuta una secuencia de validaciones criptográficas. En la Figura 4 se detalla este proceso.

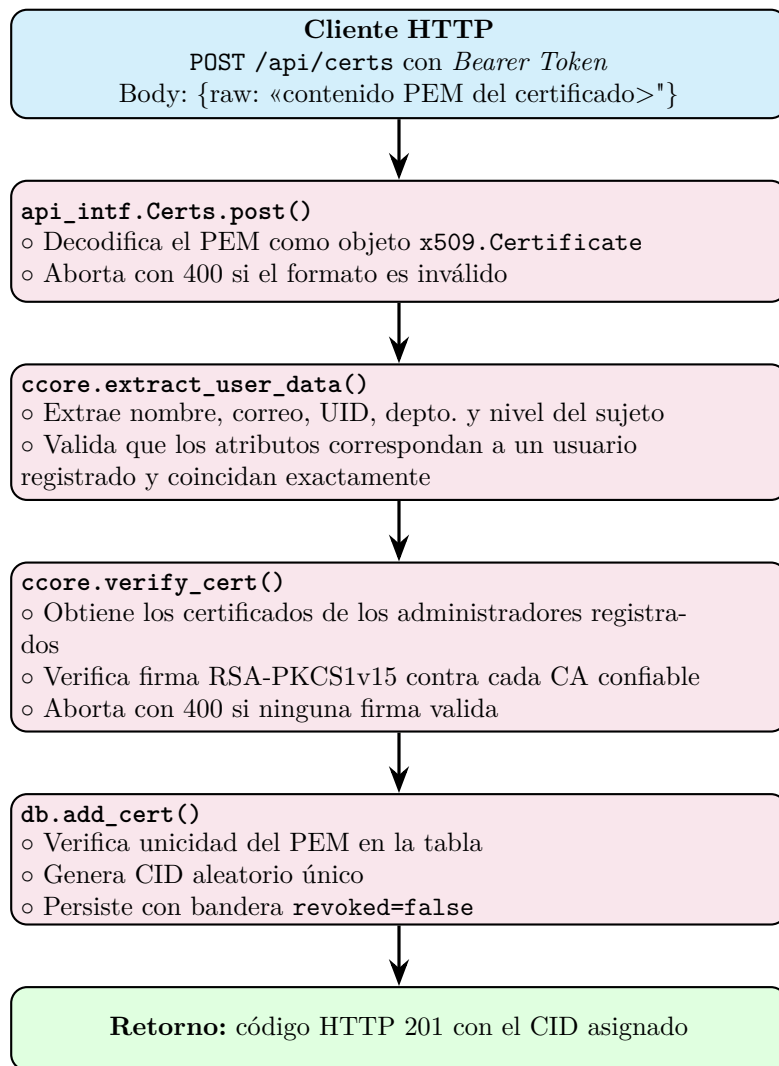


Figura 4: Diagrama de flujo: registro de un certificado X.509 emitido externamente por la CA interna (elaboración propia).

Posteriormente, las peticiones de consulta sobre certificados son atendidas por el endpoint `GET /api/certs`, que ofrece filtrado por identificador, usuario, vigencia temporal, estado de revocación y rango de fechas. La Figura 5 expone la mecánica del procesamiento de estas consultas.

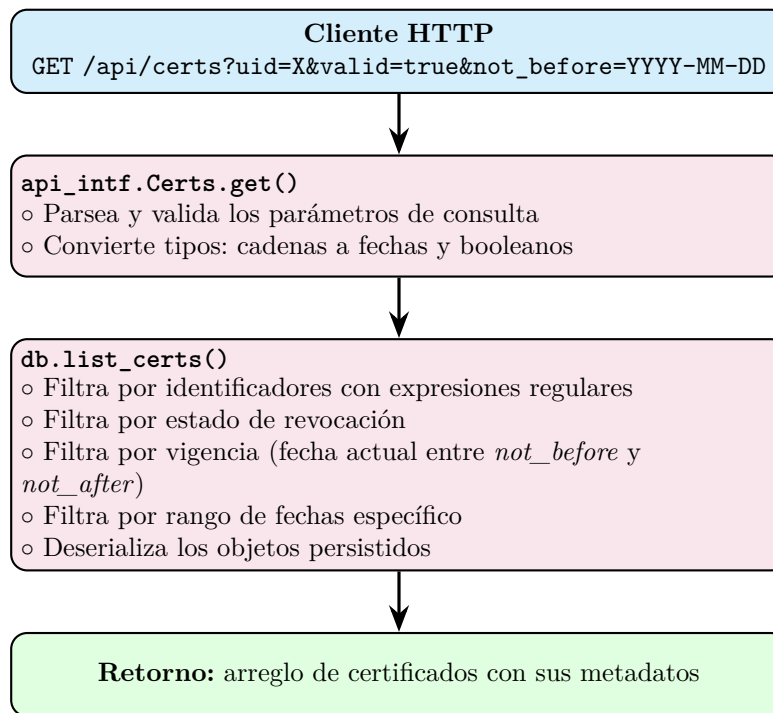


Figura 5: Diagrama de flujo: consulta y filtrado de certificados mediante el endpoint `GET /api/certs` (elaboración propia).

6.1.3. Autenticación y cambio obligatorio de contraseña

El flujo de autenticación cubre dos escenarios distintos pero relacionados: el inicio de sesión rutinario de un usuario cuya contraseña ya fue personalizada, y el primer acceso de un usuario que aún opera con una contraseña preasignada (la del administrador inicial o la temporal generada en el momento de su registro). El sistema discrimina entre ambos escenarios mediante la bandera `must_change_password` almacenada en la tabla `auth`.

Cuando el cliente envía sus credenciales al endpoint `POST /api/auth/login`, el módulo `auth.py` ejecuta la verificación contra el hash bcrypt registrado. Una verificación exitosa emite un token JWT firmado con HS256 y vigencia de ocho horas, cuyo *payload* incluye el identificador del usuario, su nivel jerárquico y su departamento. El cliente recibe el token junto con la bandera de cambio obligatorio y la utiliza para decidir el destino: si la bandera está activa, redirige al usuario a la pantalla de cambio de contraseña. Si no está activa, redirige al panel principal. La Figura 6 ilustra esta secuencia.

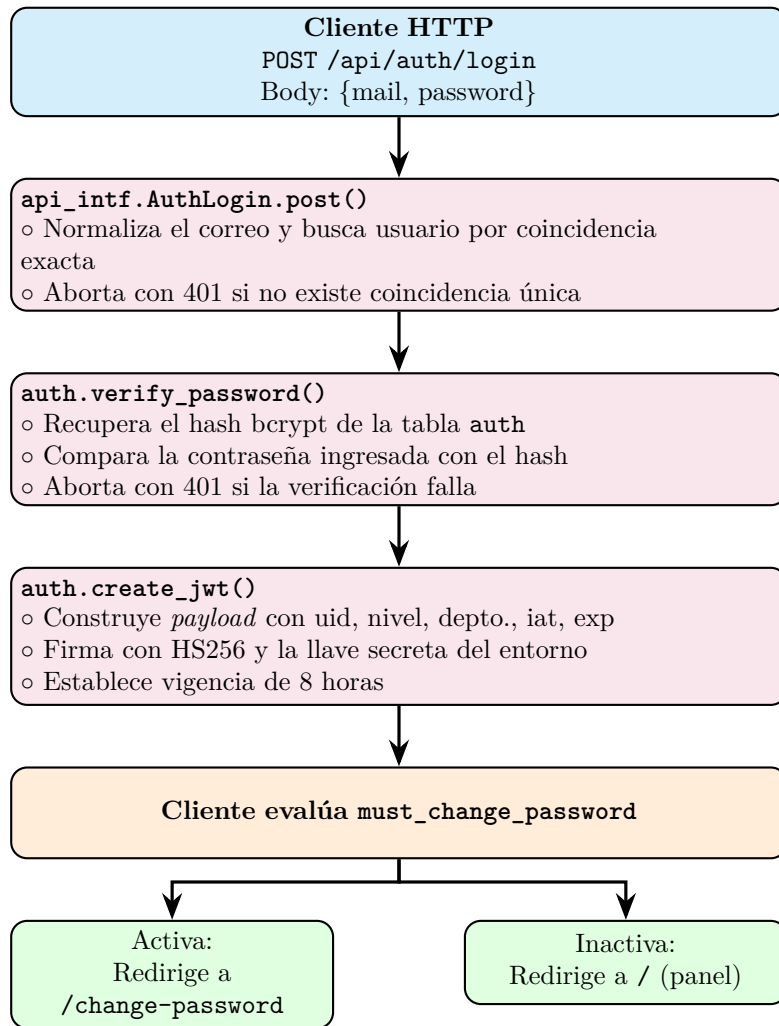


Figura 6: Diagrama de flujo: autenticación de usuario con discriminación de primer acceso (elaboración propia).

La pantalla de cambio de contraseña, una vez alcanzada, aplica cuatro validaciones independientes antes de transmitir la solicitud al endpoint POST /api/auth/change-password: la presencia de la contraseña actual, la longitud mínima de ocho caracteres en la nueva, la distinción entre la nueva y la actual, y la coincidencia entre la nueva y su confirmación. Tras la validación, el backend verifica nuevamente la contraseña actual contra el hash registrado, calcula el hash bcrypt de la nueva contraseña, actualiza la tabla `auth` y elimina la bandera de cambio obligatorio. El usuario es entonces redirigido al panel principal con su sesión ya plenamente operativa.

6.1.4. Firma y verificación pública de correos institucionales

El flujo de firma y verificación pública de correos constituye la funcionalidad más distintiva del sistema en la presente etapa. Combina criptografía del lado cliente para la firma, validación criptográfica del lado servidor para la verificación, y un mecanismo de tokens efímeros para permitir que destinatarios externos confirmen la autenticidad de un mensaje sin requerimientos técnicos adicionales.

El proceso comienza en el formulario de redacción del panel principal, donde el remitente captura el destinatario, el asunto, el cuerpo del mensaje, sus archivos adjuntos opcionales y su llave privada en formato PEM. La interfaz canonicaliza localmente el cuerpo del mensaje (normalizando los saltos de línea a Unix y ensamblando el encabezado fijo con campos `To` y `Subject`), calcula la firma RSA-PKCS1v15 con SHA-256 sobre el resultado utilizando la biblioteca `jsrsasign`, y firma adicionalmente cada archivo adjunto de manera individual. La llave privada nunca abandona el

navegador del firmante. El cliente envía al endpoint `POST /api/mail` el contenido del mensaje junto con las firmas en formato Base64.

El backend, mediante `api_intf.MailResource.post()`, verifica que el remitente tenga nivel administrador o coordinador, que posea un certificado vigente y no revocado, y que las firmas recibidas validen correctamente contra dicho certificado utilizando `mail_crypto.verify_bytes`. Si todas las verificaciones son exitosas, el sistema discrimina el tipo de destinatario: si el correo del destinatario pertenece a un usuario registrado, el mensaje se persiste sin token de verificación pública (verificable solo desde el panel autenticado), pero si el destinatario es externo, el sistema genera un token criptográficamente aleatorio de 128 bits, establece su expiración a 24 horas, y lo asocia al mensaje. La Figura 7 sintetiza la secuencia.

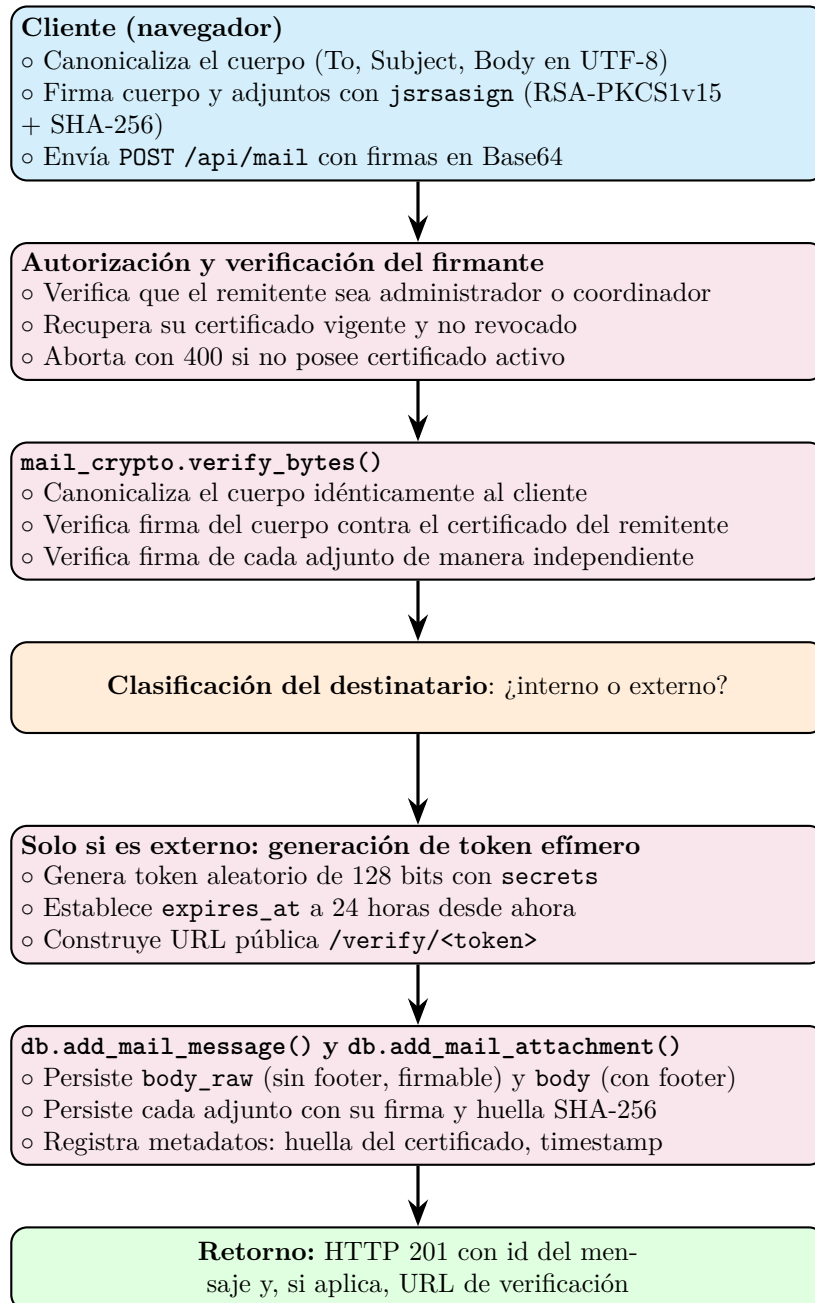


Figura 7: Diagrama de flujo: firma de un correo institucional y generación condicional del token efímero de verificación (elaboración propia).

Cuando el destinatario externo recibe el correo y abre el enlace `/verify/<token>`, el navegador carga la plantilla pública `verify.html`, la cual inmediatamente invoca al endpoint `GET`

`/api/public/verify/<token>`. El backend ejecuta una secuencia particular de pasos en la cual el consumo del token precede a la verificación criptográfica, en concordancia con un modelo de confianza cero: si el token no existe, ha expirado o ya fue consumido, el sistema responde con un código de error (404 o 410 según el caso) sin exponer información alguna del mensaje original. Esta separación entre estado del token y contenido del mensaje protege la confidencialidad del correo ante accesos no contemplados.

Si el token es válido y aún no consumido, el backend lo marca como utilizado antes de proceder con la verificación criptográfica. A continuación recupera el certificado firmante asociado al mensaje, canonicaliza el cuerpo original (almacenado como `body_raw` en la base de datos, sin el pie de página que incluye la propia URL de verificación), y verifica la firma. Adicionalmente, evalúa el estado del certificado: vigente, no revocado y dentro de su período de validez. El resultado se devuelve al cliente como un objeto estructurado con indicadores individuales por cada componente verificado, sus metadatos asociados (asunto, destinatario, identificador del firmante, huella SHA-256 del certificado) y una cadena de texto descriptiva. La plantilla `verify.html` renderiza este resultado en una tarjeta de verificación con códigos visuales diferenciados según el veredicto.

La separación entre el campo `body_raw` (sobre el cual se firma) y el campo `body` (que incluye el pie con la URL de verificación, visible para el lector pero excluido de la firma) resuelve una circularidad inherente: la URL de verificación no puede formar parte del cuerpo firmado porque depende de un token aleatorio que solo existe después de persistir el mensaje; por lo que firmar y persistir deben ocurrir antes de generar el pie. Esta arquitectura permite mostrar al destinatario un correo cuya autenticidad puede verificar sin alterar el contenido original sobre el cual se calculó la firma.

7. Stack tecnológico

Componente	Tecnología	Responsabilidad
Backend	Python + Flask	Exposición de API HTTP, reglas de negocio, autenticación y coordinación del sistema.
API REST	Flask-RESTX	Modelado de esquemas, documentación, serialización y control de endpoints.
Persistencia	TinyDB	Almacenamiento local basado en JSON para usuarios, certificados, auth y correo.
Criptografía X.509	cryptography	Generación de llaves RSA, certificados, serialización PEM y validación.
Autenticación	JWT + bcrypt	Emisión de tokens de sesión, hash seguro de contraseñas y verificación.
Frontend	HTML/CSS/JS	Panel administrativo, formularios, renderizado y firma en navegador.
Firma en cliente	jsrsasign	Firma RSA SHA-256 de cuerpo y adjuntos en el navegador.
Validación de correos	validators	Verificación de formato de correo en backend.

8. Estructura del SDK

```
src/
  api_intf.py # Endpoints REST y modelos de intercambio
  auth.py # JWT, hash de contraseñas y flujo de login
  ccore.py # Emisión y validación de certificados X.509
  db.py # TinyDB, serialización y persistencia
  mail_crypto.py # Canonicalización, firma y verificación de correo
  templates/
```

```
index.html # Frontend completo con dashboard y modulo de correo
README.md # Guia de instalacion y operacion
```

9. Modelo de dominios

9.1. Usuario

El usuario es la entidad base del sistema. Contiene identidad organizacional, clasificación jerárquica y datos mínimos para autenticación y autorización.

Campo	Tipo	Descripción
id	string	UID permanente del usuario.
name	string	Nombre completo.
dept	enum	Departamento organizacional.
lvl	enum	Nivel de acceso.
mail	string	Correo institucional único.
joined	date	Fecha de incorporación.

9.2. Departamentos soportados

- Humanitaria
- PsicoSocial
- Legal
- Comunicación
- Almacén
- TI

9.3. Niveles de acceso

En backend, el enum de niveles define los valores: **admin**, **coordinador**, **operativo** y **captura**. La documentación funcional también los describe como N1 a N4.

Nivel	Valor	Capacidad operativa
N1	admin	Control total del sistema, usuarios, certificados y visibilidad completa de correos.
N2	coordinador	Gestión limitada de usuarios operativos/captura de su mismo departamento; acceso a correo propio enviado/recibido.
N3	operativo	Consulta restringida; sin privilegios de creación administrativa.
N4	captura	Captura inicial de información; sin gestión avanzada ni firma institucional.

9.4. Certificado

Campo	Tipo	Descripción
id	string	CID del certificado.

uid	string	Usuario propietario.
notbefore	date	Inicio de vigencia.
notafter	date	Fin de vigencia.
revoked	bool	Estado de revocación.
cert/raw	X.509/PEM	Certificado en memoria y/o serializado en PEM.

9.5. Registro de autenticación

Campo	Tipo	Descripción
uid	string	Referencia al usuario.
passwordhash	string	Hash bcrypt de la contraseña.
mustchange password	bool	Fuerza cambio de contraseña en primer acceso.
createdat	string	Marca temporal de alta.
updatedat	string	Última actualización.

9.6. Correo firmado

Campo	Tipo	Descripción
id	string	Identificador del mensaje.
senderuid	string	UID del remitente.
recipient	string	Correo de destino.
subject	string	Asunto.
body	string	Cuerpo persistido, potencialmente con texto informativo para verificación.
bodyraw	string	Cuerpo original usado para verificación criptográfica.
bodysig	string	Firma detached del cuerpo canónico.
certid	string	Certificado usado para firmar.
signingcertfingerprint	string	Fingerprint SHA-256 del certificado firmante.
validationtoken	string	Token efímero para verificación pública.
expiresat	string	Expiración de token en UTC.
tokenused	bool	Indica si el token ya fue consumido.

9.7. Adjunto firmado

Campo	Tipo	Descripción
id	string	Identificador del adjunto.
messageid	string	Referencia al mensaje.
filename	string	Nombre del archivo.
mime	string	Tipo MIME.
contentb64	string	Contenido en base64.
sigb64	string	Firma detached del adjunto.
sha256	string	Digest SHA-256 del binario.

10. Persistencia y serialización

La capa `db.py` abstrae toda operación de lectura y escritura hacia TinyDB. Esta capa es crítica porque el sistema necesita persistir tipos de datos que JSON no soporta nativamente, como fechas y certificados X.509.

10.1. Serialización de fechas

Las fechas se serializan como tuplas (año, mes, día) para almacenamiento. Al leer datos, el sistema reconstruye instancias `date`.

10.2. Serialización de certificados

Mediante `public_bytes(serialization.Encoding.PEM).decode()` los certificados se convierten a texto PEM, y al deserializar se reconstruyen con `x509.load_pem_x509_certificate()`.

10.3. Tablas TinyDB

- `users`
- `certs`
- `auth`
- `mailmessages`
- `mailattachments`

11. Criptografía aplicada

11.1. Módulo `ccore.py`

El núcleo criptográfico genera certificados y procesa la identidad X.509. Sus responsabilidades incluyen:

- extracción de los atributos del usuario;
- verificación de la firma usando la llave pública por cada certificado;

11.2. Esquema criptográfico

- Algoritmo de llave pública: RSA.
- Exponente público: 65537.
- Tamaño mínimo permitido por la API: 2048 bits.
- En flujo de demostración/uso reciente, la interfaz muestra y usa certificados RSA de 4096 bits.
- Hash para firma: SHA-256.
- Formato de certificado: X.509 en PEM.

11.3. Módulo `mail_crypto.py`

Este módulo implementa primitivos de firma y verificación para correo firmado:

1. Canonicalización del cuerpo.
2. Verificación detached de firma del cuerpo.
3. Verificación detached de adjuntos.
4. Cálculo de fingerprint SHA-256 de certificados.
5. Conversión desde/hacia base64 para adjuntos.

11.4. Canonicalización del cuerpo

La firma de correo no se realiza sobre texto arbitrario tal como fue escrito visualmente, sino sobre una representación canónica estable. Conceptualmente, se arma el texto con estructura:

```
To: <recipient>
Subject: <subject>

<body-normalizado>
```

Esta decisión garantiza que cliente y servidor verifiquen exactamente la misma secuencia de bytes. La implementación del frontend replica explícitamente la canonicalización del backend para evitar divergencias de saltos de línea o espacios accidentales.

12. Autenticación

12.1. Módulo `auth.py`

El módulo de autenticación es responsable de:

- generar contraseñas temporales para nuevos usuarios;
- aplicar hash seguro a contraseñas;
- verificar contraseñas ingresadas;
- crear JWT de sesión;
- decodificar y validar JWT;
- forzar cambio de contraseña en primer acceso.

12.2. Flujo de autenticación

1. El usuario envía correo y contraseña a `/auth/login`.
2. El backend busca un usuario por correo.
3. Recupera el registro de autenticación asociado.
4. Valida la contraseña contra el hash almacenado.
5. Si es válida, genera un JWT con al menos `uid`, `lvl` y `dept`.
6. El frontend guarda el token en `sessionStorage`.
7. Las peticiones subsecuentes envían el encabezado `Authorization: Bearer <token>`.

12.3. Primer acceso

Si el registro de autenticación tiene `mustchangepassword = true`, el sistema obliga al usuario a cambiar su contraseña antes de operar normalmente.

12.4. Bootstrap administrativo

Si no existen credenciales en la tabla `auth`, el backend inicializa automáticamente un usuario administrador con correo `admin@casamonarca.mx` y contraseña temporal `admin123`. Este flujo debe considerarse únicamente de inicialización controlada y no de operación productiva final.

13. Autorización RBAC

La autorización se aplica tanto en backend como en frontend. El frontend oculta controles no permitidos; el backend rechaza operaciones no autorizadas con código HTTP 403.

13.1. Matriz de permisos

Rol	Crear usuarios	Editar usuarios	Eliminar usuarios	Correo firmado
Administrador	Sí	Sí	Sí	Sí
Coordinador	Solo operativo/ captura	Solo operativo/ captura de su depto	No	Sí, sobre sus correos
Operativo	No	No	No	No
Captura	No	No	No	No

13.2. Reglas específicas

- Un coordinador no puede crear usuarios `admin` ni `coordinador`.
- Un coordinador solo puede editar usuarios de su mismo departamento.
- Los operativos y captura no tienen permisos administrativos.
- Para firmar correo, el usuario debe ser `admin` o `coordinador` y tener un certificado activo, vigente y no revocado.

14. Contratos de API

La API usa principalmente JSON y se expone bajo una ruta base configurable. En la interfaz web, el frontend usa como base `http://localhost:8000/api`. Los ejemplos a continuación usan esa base.

14.1. Convenciones generales

- Formato de fecha: YYYY-MM-DD.
- Autenticación: encabezado `Authorization: Bearer <JWT>` cuando aplica.
- Respuestas de error: pueden emitirse con `api.abort(...)` y contener mensaje textual.
- Códigos relevantes: 200, 201, 400, 401, 403, 404, 410 y 500.

14.2. Endpoint POST `/auth/login`

Autentica al usuario y devuelve token de sesión.

Request

```
{
  "mail": "admin@casamonarca.mx",
  "password": "admin123"
}
```

Response 200

```
{
  "token": "<jwt>",
  "mustchangepassword": true,
  "uid": "1032352645",
  "name": "Administrador Casa Monarca",
  "lvl": "admin",
  "dept": "TI"
}
```

14.3. Endpoint GET /auth/me

Devuelve el perfil autenticado y el estado de cambio obligatorio de contraseña.

14.4. Endpoint POST /auth/change-password**Request**

```
{
  "oldpassword": "admin123",
  "newpassword": "Monarca2026!"
}
```

Response 200

```
{
  "status": "ok"
}
```

14.5. Endpoint GET /users

Consulta usuarios con filtros opcionales.

Parámetros soportados

- uid
- name
- dept
- lvl
- mail
- joinedbefore
- joinedafter

Response 200

```
[
  {
    "id": "1032352645",
    "name": "Administrador Casa Monarca",
    "dept": "TI",
    "lvl": "admin",
    "mail": "admin@casamonarca.mx",
    "joined": "2026-06-10"
  }
]
```

14.6. Endpoint POST /users

Crea un usuario. Requiere autenticación.

Request

```
{
  "name": "Maria Lopez",
  "dept": "Legal",
  "lvl": "coordinador",
  "mail": "maria.lopez@casamonarca.mx",
  "joined": "2026-06-10"
}
```

Response 200

```
{
  "id": "208745901",
  "temppassword": "A7Q2X9K4M1P8..."
}
```

14.7. Endpoint PATCH /users

Actualiza nombre, departamento, nivel, correo o fecha de ingreso de un usuario, respetando la política RBAC.

14.8. Endpoint DELETE /users

Elimina un usuario. Solo disponible para administrador.

14.9. Endpoint GET /certs

Consulta certificados por filtros.

Parámetros soportados

- cid
- uid
- valid = true|false
- notbefore = YYYY-MM-DD
- notafter = YYYY-MM-DD
- revoked = true|false

14.10. Endpoint POST /certs

Registra un certificado PEM ya emitido. El backend valida que:

- el PEM sea un certificado válido;
- los datos del certificado correspondan exactamente a un usuario registrado;
- el certificado esté firmado por un administrador registrado;
- no exista ya el mismo certificado en la base.

Request

```
{
  "raw": "-----BEGIN CERTIFICATE-----\nMIIC...\n-----END CERTIFICATE-----"
}
```

Response 200/201 El registro no devuelve un esquema complejo; basta con considerar la operación exitosa si no hay error.

14.11. Endpoint GET /certs

Devuelve el PEM del certificado específico.

Response

```
{
  "raw": "-----BEGIN CERTIFICATE-----\nMIIC...\n-----END CERTIFICATE-----"
}
```

14.12. Endpoint GET /mail

Lista correos firmados almacenados. El frontend posteriormente consulta su verificación uno a uno.

14.13. Endpoint POST /mail

Crea un correo firmado cuyo cuerpo y adjuntos ya vienen firmados desde el cliente. El backend solo verifica y persiste.

Request

```
{
  "uid": "208745901",
  "recipient": "receptor@externo.org",
  "subject": "Notificacion oficial",
  "body": "Contenido del correo.",
  "bodysig": "<firma-base64>",
  "attachments": [
    {
      "filename": "oficio.pdf",
      "mime": "application/pdf",
      "contentb64": "<base64>",
      "sig": "<firma-base64>"
    }
  ]
}
```

Lógica interna

1. Verifica que el usuario exista.
2. Restringe el uso a **admin** o **coordinador**.
3. Obtiene un certificado activo, vigente y no revocado.
4. Reconstruye el cuerpo canónico.
5. Verifica **bodysig** con el certificado registrado.
6. Verifica cada adjunto con la misma credencial.

7. Si el destinatario es externo, genera token efímero de 24 horas y URL pública.
8. Persiste mensaje y adjuntos.

Response 201

```
{
  "id": "15",
  "validationtoken": "8D7E4A...",
  "validationurl": "http://localhost:8000/api/public/verify/8D7E4A..."
}
```

14.14. Endpoint POST /mail/x509

Variante que recibe la clave privada PEM en el request para firmar en servidor o en flujo especial de integración. Aunque existe, para máxima seguridad se recomienda preferir el flujo principal donde la firma se calcula en el navegador y la clave nunca sale del equipo del usuario.

14.15. Endpoint GET /mail/<messageid>/verify

Verifica un correo almacenado en contexto autenticado.

Response

```
{
  "messageok": true,
  "attachmentsok": true,
  "certificateok": true,
  "signeruid": "208745901",
  "fingerprint": "ABCD1234...",
  "details": "Valid signature"
}
```

14.16. Endpoint GET /public/verify

Verificación pública de un solo uso para destinatarios externos.

Comportamiento

- Si el token no existe: 404.
- Si ya fue usado: 410.
- Si expiró: 410.
- Si está vigente: consume el token, verifica el mensaje y devuelve el resultado.

Response

```
{
  "messageok": true,
  "certificateok": true,
  "signeruid": "208745901",
  "fingerprint": "ABCD1234...",
  "subject": "Notificacion oficial",
  "recipient": "receptor@externo.org",
  "createdat": "2026-06-10T23:00:00Z",
  "details": "Firma criptografica valida. El contenido no ha sido alterado."
}
```


15. Frontend SDK

15.1. Estado local

El frontend mantiene estructuras de estado en memoria:

- USERS
- CERTS
- MAILS
- MAILVERIFICATIONS
- índices derivados como `certsByUid`

15.2. Guardia de autenticación

Al cargar la aplicación, se valida que exista un token y un usuario serializado en `sessionStorage`. Si no existen, el navegador redirige a `/login`.

15.3. Cliente HTTP genérico

La función `apiFetch(path, opts)` agrega automáticamente el JWT al encabezado `Authorization`. Además, si el backend devuelve 401, limpia la sesión y redirige al login.

15.4. Firma en navegador

El frontend incorpora `jsrsasign` y aplica la secuencia:

1. construir cuerpo canónico;
2. convertir texto a bytes/hex UTF-8;
3. firmar con RSA PKCS#1 v1.5 y SHA-256;
4. firmar cada adjunto sobre bytes binarios crudos;
5. enviar firmas detached al backend.

Pseudocódigo del flujo cliente

```
def enviar_correo_firmado(uid, recipient, subject, body, private_key_pem, archivos):
    canonical = canonicalize_body(subject, recipient, body)
    body_sig = sign_rsa_sha256(private_key_pem, canonical)
    attachments = []
    for archivo in archivos:
        raw = archivo.read_bytes()
        sig = sign_rsa_sha256(private_key_pem, raw)
        attachments.append({
            "filename": archivo.name,
            "mime": archivo.mime,
            "contentb64": b64(raw),
            "sig": sig,
        })
    payload = {
        "uid": uid,
        "recipient": recipient,
        "subject": subject,
        "body": body,
        "bodysig": body_sig,
        "attachments": attachments,
```

```
}  
return POST("/mail", payload)
```

16. Flujos principales

16.1. Flujo A: alta de usuario

1. Un administrador o coordinador autenticado abre el modal de registro.
2. El frontend valida nombre, correo y nivel.
3. Envía POST /users.
4. El backend valida correo, permiso del actor y unicidad.
5. Se asigna UID aleatorio único.
6. Se genera contraseña temporal.
7. Se almacena hash bcrypt en tabla auth.
8. El frontend muestra la contraseña una sola vez.

16.2. Flujo B: carga o generación de certificado

1. El administrador obtiene o genera el PEM del usuario.
2. Puede usar POST /users/<uid>/generatecert en entorno de prueba.
3. Alternativamente, arrastra un archivo PEM y usa POST /certs.
4. El backend verifica correspondencia exacta entre subject y usuario registrado.
5. El certificado queda asociado al UID.

16.3. Flujo C: login y sesión

1. El usuario envía credenciales a /auth/login.
2. El backend verifica hash y emite JWT.
3. El frontend guarda token y perfil.
4. Si debe cambiar contraseña, lo redirige al flujo obligatorio.

16.4. Flujo D: correo firmado interno/externo

1. El usuario N1/N2 abre el modal de correo.
2. La firma se calcula localmente con la clave PEM del remitente.
3. El payload se envía a /mail.
4. El backend verifica firmas con el certificado oficial del remitente.
5. Si el destinatario no pertenece al gestor, se crea token efímero de 24 horas.
6. El mensaje se persiste en mailmessages y sus adjuntos en mailattachments.

16.5. Flujo E: verificación pública

1. Un tercero abre la URL `/public/verify/<token>`.
2. El backend valida existencia, no reutilización y expiración.
3. Marca el token como usado antes de verificar el contenido.
4. Verifica firma del cuerpo, vigencia del certificado y estado de revocación.
5. Devuelve un veredicto legible y criptográficamente consistente.

17. Ejemplos de integración en Python

17.1. Login

```
import requests

BASE = "http://localhost:8000/api"

resp = requests.post(f"{BASE}/auth/login", json={
    "mail": "admin@casamonarca.mx",
    "password": "admin123"
})
resp.raise_for_status()
data = resp.json()
token = data["token"]
headers = {"Authorization": f"Bearer {token}"}
```

17.2. Crear usuario

```
resp = requests.post(f"{BASE}/users", headers=headers, json={
    "name": "Maria Lopez",
    "dept": "Legal",
    "lvl": "coordinador",
    "mail": "maria.lopez@casamonarca.mx",
    "joined": "2026-06-10"
})
resp.raise_for_status()
print(resp.json())
```

17.3. Consultar certificados vigentes

```
resp = requests.get(f"{BASE}/certs", headers=headers, params={
    "valid": "true",
    "revoked": "false"
})
resp.raise_for_status()
certs = resp.json()
```

17.4. Verificar un correo por ID

```
message_id = "15"
resp = requests.get(f"{BASE}/mail/{message_id}/verify", headers=headers)
resp.raise_for_status()
print(resp.json())
```

18. Ejemplos de integración en JavaScript

18.1. Consumir la API autenticada

```
async function apiFetch(path, token, opts = {}) {
  const res = await fetch('http://localhost:8000/api/${path.replace(/~/, '')}', {
    ...opts,
    headers: {
      'Content-Type': 'application/json',
      'Authorization': 'Bearer ${token}',
      ...(opts.headers || {})
    }
  });
  if (!res.ok) throw new Error(await res.text());
  return await res.json();
}
```

18.2. Login en JavaScript

```
const login = await apiFetch('auth/login', null, {
  method: 'POST',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({
    mail: 'admin@casamonarca.mx',
    password: 'admin123'
  })
});
const token = login.token;
```

19. Manejo de errores

19.1. Errores de validación típicos

- correo inválido;
- correo duplicado;
- fechas en formato incorrecto;
- fechas de certificado fuera de rango lógico;
- key size inferior a 2048;
- usuario no encontrado de manera única;
- credenciales inválidas;
- token inexistente, expirado o ya consumido.

19.2. Patrones recomendados para integradores

1. distinguir fallos de autenticación (401) de autorización (403);
2. mostrar mensajes amigables para errores de entrada (400);
3. manejar 410 específicamente para enlaces públicos expirados o ya utilizados;
4. registrar en logs técnicos el cuerpo de error devuelto por la API.

20. Seguridad y mejores prácticas

1. No almacenar el JWT en mecanismos persistentes de larga duración si no es necesario.
2. Mantener la clave privada PEM del firmante fuera del backend siempre que sea posible.
3. Usar HTTPS en despliegues reales.
4. Rotar la clave secreta JWT fuera del repositorio y proteger el archivo `.env`.
5. Obligar a cambio de contraseña temporal en primer acceso, como ya lo hace el sistema.
6. No reutilizar tokens de verificación pública; el diseño actual ya aplica un solo uso.
7. Revisar la persistencia de revocación para hacerla completamente duradera en backend si se expande la CRL.

21. Limitaciones actuales y extensiones sugeridas

- La persistencia se basa en TinyDB; para mayor concurrencia conviene migrar a motor relacional.
- La revocación de certificados aparece parcialmente modelada; una versión de producción debe exponer un endpoint formal de revocación persistente.
- El endpoint `/mail/x509` existe, pero la ruta principal recomendada sigue siendo la firma local en cliente.
- Puede integrarse MFA sobre el flujo de autenticación JWT.
- El sistema es apto para extenderse a firma de documentos y no solo correo.

Referencias

- [Grassi et al., 2017] Grassi, P. A., Garcia, M. E., and Fenton, J. L. (2017). Digital identity guidelines. Technical Report Special Publication (NIST SP) 800-63-3, National Institute of Standards and Technology (NIST).
- [Stallings, 2017] Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice*. Pearson, 7th edition.
- [Zúñiga and Hecht, 2023] Zúñiga, A. and Hecht, F. (2023). Mecanismos de autorización dinámicos y granulares en sistemas modernos. *Revista de Seguridad Informática y Sistemas*.

22. Apéndice: esquema resumido de endpoints

Método	Ruta	Descripción
GET	/users	Lista usuarios con filtros.
POST	/users	Crea usuario y contraseña temporal.
PATCH	/users?uid=...	Edita usuario.
DELETE	/users?uid=...	Elimina usuario.
GET	/certs	Lista certificados.
POST	/certs	Registra un PEM de certificado.
GET	/certs/<cid>	Devuelve PEM del certificado.
POST	/users/<uid>/generatecert	Genera certificado demo.
POST	/auth/login	Inicia sesión.
GET	/auth/me	Consulta usuario autenticado.
POST	/auth/change-password	Cambia contraseña.
POST	/auth/logout	Cierre lógico de sesión.
GET	/mail	Lista correos firmados.
POST	/mail	Crea correo firmado verificando firmas.
POST	/mail/x509	Variante de firmado con PEM.
GET	/mail/<id>/verify	Verifica correo autenticado.
GET	/public/verify/<token>	Verificación pública de un solo uso.

23. Apéndice: recomendaciones para Overleaf

Para compilar este documento en Overleaf:

1. Crear un proyecto nuevo vacío.
2. Pegar este contenido en un archivo `main.tex`.
3. Compilar con pdfLaTeX.
4. Si se desean fragmentos de código más extensos, mantener `listings` o migrar a `minted` si el proyecto habilita `-shell-escape`.